anime-rag

Multimodal (Agentic) RAG over the Manga109-s manga dataset. Individual NLP course project.

- **Code:** https://github.com/martin76ec/manga-multi-modal-rag
- **Built index (embeddings + metadata, no images):** https://huggingface.co/datasets/martin76ec/Manga109-s-indexes
- **Demo video:** https://www.youtube.com/watch?v=bUYKq-7YVvg

Course project overview

Topic

RAG / Agentic RAG. Not built with LangChain/LangGraph. This is a deliberate, tested deviation, not an oversight:

- `google/gemma-4-26B-A4B-it` (needed for vision, so the agent can judge a panel directly, not just via translated dialogue) does not reliably use native tool-calling. Confirmed with the raw HF Inference API, bypassing LangChain entirely: given a registered tool, it just answered in plain text claiming it had none.
- Wiring it into LangGraph's `create_react_agent` was worse: it hallucinated a complete fake tool call, with a fabricated URL and fictional data.
- Models that do tool-call reliably on the free providers I have access to (`Llama-3.3-70B`, `Qwen2.5-72B`) are text-only. The one vision+tool-calling model I found (`Qwen2.5-VL-32B`) also ignored the tool and hallucinated an answer.

No model available to me does both reliably at once. The hand-built agent in `agent.py` (MRKL-style `Thought → Action → Observation → Final Answer`, plain-text prompted, with retry/degeneration detection added after real provider failures) is the result of testing LangChain and finding it less reliable here, not a workaround for skipping it.

Problem and data

Manga dialogue in Manga109-s is Japanese-only, and panels carry visual information the transcribed text often doesn't describe. Goal: let someone who doesn't read Japanese ask a question in English and get an answer grounded in cited panels, using both artwork and dialogue.

Data: Manga109-s (gated), 87 licensed manga volumes with page images and per-panel annotations. Fully indexed: **87 books, 82,124 panels**.

Baseline vs. proposed solution

	Baseline	Proposed
Retrieval	single modality: text-only or image-only CLIP similarity	blended score (`0.6 image + 0.4 text` , tunable) over the same embeddings
Query language	Japanese only	English or Japanese (NLLB-200 translation bridge)
Answering	top-k panels returned directly	MRKL+IRCoT agent: multi-round, vision-grounded, cites book/page, says "I don't know" instead of guessing

Comparison methodology and results

`scripts/eval.py` now provides a small, repeatable quantitative check (`make eval`): hit-rate/MRR for retrieval (no LLM, RAGAS-style context precision/recall), and an LLM-judge faithfulness/relevancy score for the agent (RAGAS-style, using our own `ChatLLM` , not the `ragas` package, which hard-depends on LangChain internals and fails to import on the current stack). It's still small-N (2 verified cases) and development-selected, not a random sample; see Conclusions. The tables below are from development-time testing, mostly at the 2-book scale used early in the project.

Modality ablation (query: `"police officer"` , at 2-book scale):

Mode	Top-1	Score	Verified correct?
Image-only	`AisazuNihaIrarenai_007`	0.256	not independently verified, weaker signal
Text-only	`AisazuNihaIrarenai_057`	0.740	yes, panel reads "ちょっと警察の方!!"
Blended (default)	`AisazuNihaIrarenai_057`	0.403 (`0.6x0.178 + 0.4x0.740`)	yes, checked by hand

Isolating either modality changes the ranking. Text carried more signal here; the blend keeps results robust for queries where the opposite holds (e.g. a purely visual one). Re-run at full scale (87 books) via `scripts/eval.py` , the blend still wins outright: sweeping `w_img` from 0.0 to 1.0 gives hit-rate 1.0 only at the default 0.6/0.4 (and 0.8/0.2), versus 0.0-0.5 everywhere else, with MRR peaking exactly at the default.

Baseline vs. agent:

Query	Baseline	Agent
"police officer"	correct top-1, single shot	same, 1 round
"What happens when a police officer shows up at someone's house?"	needs the user to already know the right search term	finds and cites the same panel unprompted, with a grounded answer
"someone holding a gun" (no real match indexed)	returns a plausible but wrong top result	correctly reports no match found
"someone holding a gun" (real match indexed)	finds the panel, no explanation	finds it and visually confirms it ("a Glock handgun"), a detail the text doesn't mention
multi-hop father/parenting question	can't answer at all	4 self-generated retrieval rounds, found the father, then honestly declined to confirm a mistranslated detail instead of guessing

The agent's real advantage is saying "I don't know" instead of confidently returning a wrong top-1, and chaining queries for questions that need more than one search. That costs more (next section), so it's a trade-off, not a strict upgrade.

Cost comparison

	Baseline	Agent
Where it runs	fully local	retrieval local, reasoning via HF Inference API (network, third party)
LLM calls/question	0	up to 5 (`max_steps=4` + forced final)
Tokens/call	0	`max_tokens=250` , plus growing conversation history
Images sent	0	up to 3 panels/round x 4 rounds, 384px JPEG
Latency	sub-second	tens of seconds
Monetary cost	none	HF free tier (hit a 429 rate limit once)

Local compute/storage (shared by both modes):

	Value
Panels indexed	82,124 (87 books)
Embedding throughput	~58 panels/sec (shared RTX 4060 Ti, 8GB)
Local models	CLIP Japanese (~790MB), NLLB-200 (~2.4GB), both free
Index size	380.5MB (one Parquet file)
Raw data on disk	~7.8GB

Conclusions

What worked:

- Blending image and text embeddings changes outcomes versus either alone, verified by hand.
- The translation bridge makes the system usable without Japanese, fully local, no API key.
- The agent's vision is not cosmetic: it found and described a detail ("a Glock handgun") the dialogue never mentioned.
- The agent consistently abstained instead of hallucinating a plausible wrong answer, which matters most for a citation-grounded tool.

What didn't work, fixed along the way:

- CLIP collapsed very short dialogue (e.g. "……") into a generic embedding; fixed with a minimum-text filter.
- NLLB dropped key words translating stuttering dialogue; fixed with beam search and repetition controls.
- The HF provider occasionally returned malformed output: once a fake empty tool-call, twice different repetition-loop patterns. Each required real reproduction and root-causing.
- The LangChain/LangGraph attempt failed outright for this model, a real negative result.

Limits:

- `scripts/eval.py` exists now but is tiny (2 verified cases) and development-selected, not a random or exhaustive sample.
- Recall, not precision, is the main failure mode: evidence missing from the index simply can't be found.
- Character-name translation is unreliable (no manga-specific dictionary).
- Agent reliability partly depends on third-party provider behavior outside this project's control.

AI-generated content declaration

Built with extensive assistance from Claude (Anthropic) as an engineering collaborator: architecture, implementation, debugging real failures, and this documentation. Design decisions and acceptance of each change were mine; the AI did the hands-on work under that direction, and its claims were verified against real run output, not taken at face value. Adjust this section for your institution's disclosure policy before submitting.

Presentation (5 min + 3 min Q&A)

Suggested flow: problem/data (30s), topic and the LangChain/LangGraph finding (45s), live demo of `anime-rag ask` (2 min), comparison and cost tables (1 min), conclusions/limits (45s).

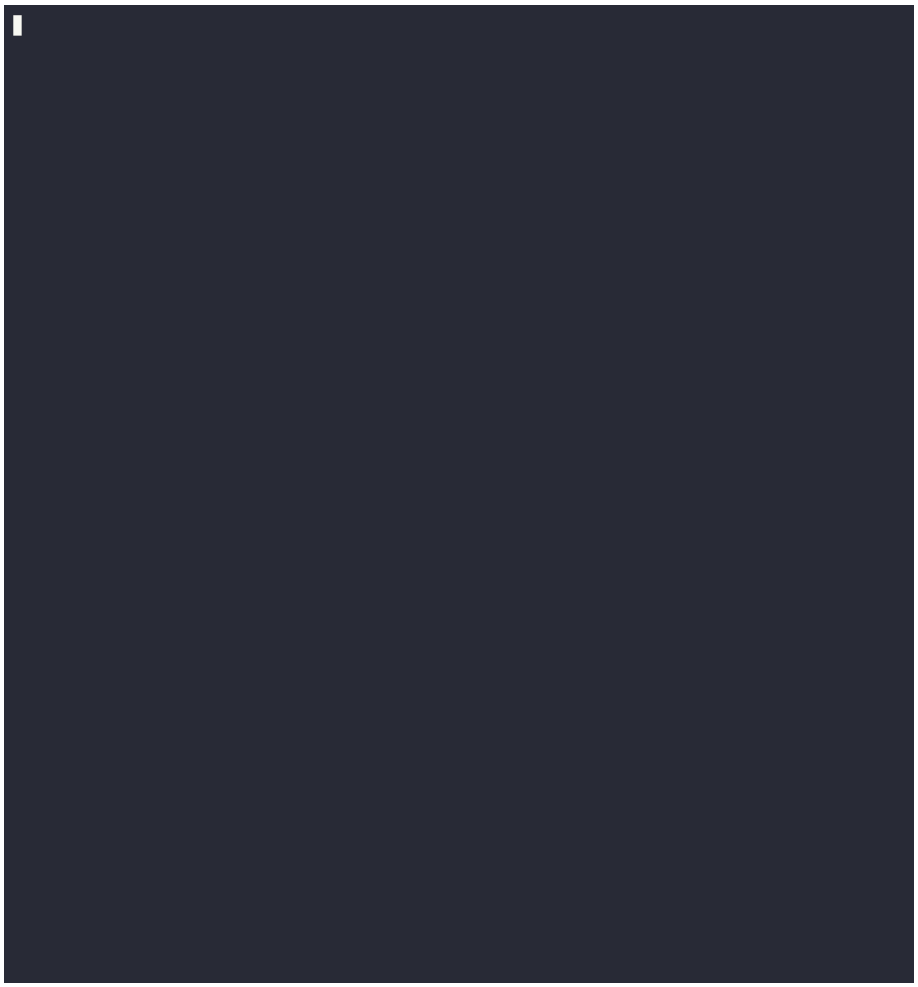
Setup

```
uv sync
hf auth login                  # needed once, for the gated dataset
```

Manga109-s is a gated dataset. Request access on the dataset page first.

Usage

```
# interactive REPL: loads models once, then take as many questions as you like.
# menu-driven (search by text / by image / ask the agent / quit), colored
# output, and an inline true-color thumbnail of the top result panel
uv run anime-rag ask
```



How it works

- `source.py` opens the dataset's zip archive through `HfFileSystem` and extracts only the requested books' images/annotations (partial HTTP range reads).
- `annotations.py` parses each book's Manga109 XML into pages of frames (panels), text boxes, and character face/body boxes.
- `panels.py` crops one image per frame and associates it with the dialogue text and character names whose boxes fall inside that frame.
- `embedder.py` wraps the Japanese CLIP model to embed panel crops and text into a shared space.
- `translate.py` translates English queries to Japanese before embedding, and Japanese result dialogue back to English for display, via `facebook/nllb-200-distilled-600M` (local, no API key needed).
- `indexing.py` builds and persists a per-book index (embeddings + metadata) under `data/index/`.
- `search.py` ranks panels for a query by a weighted blend of image-embedding and text-embedding cosine similarity. Panels whose dialogue is empty or too short to embed meaningfully (e.g. "……") fall back to image-only scoring.
- `agent.py` implements a **MRKL + IRCoT** retrieval agent on top of `search.py`: a MRKL-style explicit `Thought → Action → Observation → Final Answer` transcript (`llm.py`, via the HF Inference API, the only part of this project that's not local) drives a single `search_manga` tool, but IRCoT-style, each retrieval query is the agent's own latest reasoning step rather than a fixed query, so it can chase multi-hop questions across several retrieval rounds and still ground its answer in cited book/page evidence (or admit it doesn't know, instead of guessing). The LLM (`google/gemma-4-26B-A4B-it`) is vision-capable, so each Observation includes the actual panel images, not just translated dialogue; it can judge what's visually happening in a panel even when the dialogue doesn't describe it.

Downloaded data and built indices live under `data/` (gitignored). The dataset license forbids redistribution.

`anime-rag agent` calls the Hugging Face Inference API (model: `google/gemma-4-26B-A4B-it` by default) and needs network access plus an HF token with inference access (`hf auth login`); everything else in this project (embedding, translation, indexing) runs fully locally.

Indexing the full dataset (e.g. on a bigger GPU)

The dataset has 87 books (82,124 panels total once all are indexed). Embedding that locally on a small/shared GPU takes a while, so on a bigger box (this was written with an H200 in mind):

```
# on the bigger machine: clone this repo, then
uv sync
hf auth login                       # needs Manga109-s access

# index every book in one go; bump --batch-size way up on a big GPU
uv run anime-rag index --all --batch-size 256

# or, as a single script (also pushes the result to HF when given a repo id):
./scripts/h200_index_all.sh yourname/manga109-rag-index
```

`--all` opens the dataset zip once and indexes everything it finds via `anime-rag books`, skipping any book already indexed (safe to re-run / resume after an interruption).

Moving the index to another machine

`push-index` / `pull-index` upload/download **only the built index** (embeddings + metadata) via a HF dataset repo. It never includes the actual panel crop images, which live in a separate directory (`data/raw/crops/`) the license is stricter about: this is derived numeric output, analogous to publishing a pretrained model, not a copy of the dataset itself (see the license note below). Push it **private** (this project's own index, linked above, is public by request).

The index is consolidated into a single `index.parquet` file before uploading. Parquet is the standard single-file format for embeddings + metadata (what HF's own `datasets` library uses), and far easier to move around than one directory per book (87 books = 348 small files otherwise). `pull-index` downloads that one file and transparently re-splits it back into the usual `data/index/<book>/` layout, so nothing else changes.

```
# on the box that built the index
uv run anime-rag push-index yourname/manga109-rag-index

# if you previously pushed with an older version of this tool that uploaded
# the raw per-book folders instead of a single file, add --clean once to
# remove that old structure from the repo after the parquet file is up
uv run anime-rag push-index yourname/manga109-rag-index --clean

# on another machine
uv run anime-rag pull-index yourname/manga109-rag-index

# the pulled index has no crop images yet, regenerate them locally
# (re-extracts raw pages from the dataset zip, crops them; no GPU/model
# inference involved, so it's cheap even without the bigger box)
uv run anime-rag crops
```

Search/citations work immediately after `pull-index` (it's pure embedding math); you only need `anime-rag crops` once you want to actually view a result's panel image or use the vision-capable agent.

License note

Manga109-s is distributed under its own license (non-commercial-friendly but not open), which caps how much of any one book's pages may be published and forbids redistributing the dataset itself. See the dataset page for details.